

RAPPORT GENIE LOGICIEL

Education, performance académique & niveau de vie

M1 MIAGE - IDMC

Année Scolaire : 2025/2026

SOUS LA DIRECTION DE : M. HORNBECK BRANDON

Membres du groupe :

BASSE Grégory

EL HADIF Abderrahmane

MOHAMMED Lahmar Reda

KIAZI Febbe

MAHAMAT HADJE Garba

Table des matières

PARTIE I : INTRODUCTION	4
1. Présentation rapide de l'application	4
a) Jeux de données	4
b) Fonctionnalités principales	5
2. Présentation de l'équipe	6
3. Méthode de travail	6
4. Tuto d'installation	7
a) Cloner le projet.....	7
b) Créer la base de données PostgreSQL.....	7
c) Créer l'environnement Python	7
d) Configurer les secrets	7
e) Importer les données.....	7
f) Lancer l'application	7
PARTIE II : CONCEPTION	8
1. Diagramme de classe	8
a) Règles de gestion	8
b) Diagramme de classe 1.0	9
2. Evolution de la conception	10
a) Diagramme de classe 2.0	10
b) Dictionnaire de données	11
c) Choix effectués	17
3. Autres diagrammes pertinents	18
a) Workflow général de l'application	18
b) Diagramme d'état-transition « Dataset »	19
4. Challenges de conception	19
5) Maquettes	20
PARTIE III : Description technique	23
1. Technologies utilisées	23
2. Traitement des données	25
3. Challenges techniques.....	25
PARTIE IV : APPLICATION	26

1. Présentation détaillée de l'application	27
a) L'Espace Administrateur :.....	27
b) L'Espace Utilisateur :	28
2. Perspectives	29

PARTIE I : INTRODUCTION

De nos jours, avec l'évolution de l'informatique et des nouvelles technologies, on voit tous les jours de nouvelles innovations voire le jour. L'exemple le plus parlant est l'évolution constante de l'intelligence artificielle. En tant qu'étudiants, nous sommes réellement les premiers touchés par ces innovations. Cela dit, un constat s'effectue clairement ; il est de plus en plus difficile d'innover et de concevoir des systèmes ou des solutions utiles qui n'ont pas encore vu le jour ou qui ne verront pas le jour dans pas longtemps.

Dans le cadre de notre projet de génie logiciel, nous nous sommes donc posé la question en analysant notre environnement de savoir si nous pouvions avec nos compétences de conception et de gestion de système d'information mettre sur pied une application utile et innovante. Dans ce sens, deux secteurs ont attiré notre attention, celui de l'éducation et celui des conditions de vie des étudiants/élèves.

1. Présentation rapide de l'application

L'idée principale qui nous a guidé dans la volonté de concevoir notre application était de pouvoir étudier la corrélation entre les résultats des élèves à un examen et leur condition de vie.

Le projet consiste donc à réaliser une application/plateforme interactive *ouverte* « **IDMC – EduData France** » permettant aux utilisateurs d'explorer et d'analyser des données éducatives en France à travers des visualisations graphiques et des indicateurs statistiques.

Le périmètre de notre projet s'étend sur toute la France métropolitaine et outre-mer. Il concerne les lycéens en classe de terminal et la période d'étude concerne les résultats au baccalauréat de 2021 à 2024 inclus.

a) Jeux de données

Pour pouvoir effectuer ses analyses, nous nous sommes reposés sur l'utilisation de données officielles provenant de « *datagouv* » et « *data education gouv* ». (Voir les liens vers les jeux de données en annexes).

Le premier jeu de données contient les résultats au baccalauréat par académie et par voie du baccalauréat. Il regroupe l'ensemble des candidats au baccalauréat des sessions

de juin et de septembre, quels que soient leur origine scolaire (candidats individuels compris) et le ministère délivrant le diplôme (éducation nationale ou agriculture).

Et le deuxième les indices de position sociale des lycées.

b) Fonctionnalités principales

Afin de définir les fonctionnalités principales de notre application, nous avons défini 03 principaux acteurs dans notre système d'information : **l'administrateur** de la plateforme, **l'utilisateur et le système lui-même** qui automatisera certaines fonctionnalités.

Les fonctionnalités principales sont donc :

- **L'administrateur**

- Importer les jeux de données
- Mettre à jour les données
- Démarrer le nettoyage des données (vérification des doublons, espaces vides...)
- Valider les données avant publication
- Démarrer la fusion des jeux de données
- Gérer la base de données

- **L'utilisateur**

- Consulter et filtrer les données
- Visualiser des graphiques
- Filtrer les analyses
- Comparer les académies
- Consulter les indicateurs comparatifs
- Visualiser une carte interactive
- Exporter les données

- **Le système**

- Importer automatique les données (Optionnel)
- Nettoyer et préparer les données (normalisation des données, supprimer les doublons...)
- Fusionner les jeux de données (Création d'une base de données)
- Calcul automatique des indicateurs (**taux de réussite au bac, taux de mentions, IPS moyen par académie, écarts par rapport à la moyenne nationale**)
- Analyse statistique automatique (corrélation IPS-Réussite, tendances par années.)
- Générer les visuels (tableau de bord, graphes statistiques, cartes interactives).
- Mettre à jour les tableaux de bord
- Sauvegarder les données
- Détecter les anomalies

Nous avons modélisé ces différents acteurs et fonctionnalités dans un diagrammes de cas-utilisation (**Voir annexe**).

2. Présentation de l'équipe

Notre équipe est donc composé de 05 membres :

BASSE Grégory : Chef de projet. Pilote le projet, définis les tâches, organise le travail. Il est présent dans tous les différents axes du projet de la conception au développement.

MAHAMAT HADJE Garba : Co-responsable du Design et du développement Frontend.

KIAZI Febbe : Co-responsable du Design et du développement Frontend.

MOHAMMED Lahmar Reda : Co-responsable du développement du Backend.
S'occupe d'assurer la cohérence entre notre base de données et nos données.

EL HADIF Abderrahmane : responsable du développement Backend de l'application.

3. Méthode de travail

Nous avons décidé de suivre une méthode de travail agile pour la conception de notre application afin de pouvoir gérer les différents changements et modification tout au long du projet.

Au tout début du projet, après avoir trouvé notre sujet d'étude, nous avons donc établi un cahier de charge afin d'y recensé les différentes fonctionnalités de l'application, les tâches à réaliser et les rôles des différents membres de l'équipe.

Cela dit, le travail de conception a vraiment été un travail d'équipe et pas juste une tâche confiée à un membre de l'équipe car nous avons estimé que nous devrions tous non seulement comprendre mais aussi être entièrement d'accord sur ce que fera l'application et comment nous allons implémenter tout cela.

Afin donc de pouvoir mettre en commun notre travail, nous avons effectué des réunions quotidiennes selon la complexité à réaliser de la/ des tâche(s) précédent la réunion. Ces réunions se déroulaient soit en présentiel, soit à distanciel via notre plateforme d'échange discord.

Concernant le développement backend et notre dépôt Git, notre méthode de travail a été de travailler en local sur quelques machines puis lors d'une réunion de mis en commun, après s'être mis d'accord sur une implémentation, laisser le responsable backend effectuer le dépôt afin d'éviter que ceux qui s'y connaissent moins ne « commit & push » des erreurs.

4. Tuto d'installation

Étapes à suivre pour reproduire le projet :

a) Cloner le projet

```
git clone <url-du-repo>
cd Genie-Logicielle
```

b) Créer la base de données PostgreSQL

```
psql -U postgres -c "CREATE DATABASE projet_gl;"
psql -U postgres -d projet_gl -f database/BDD.sql
```

c) Créer l'environnement Python

```
python -m venv venv
.\venv\Scripts\activate    # Windows
source venv/bin/activate   # Linux/Mac
pip install -r requirements.txt
```

d) Configurer les secrets

```
cp .env.example .env
# Éditer .env avec les vraies valeurs
```

e) Importer les données

```
cd Backend
python -m app.scripts.run_import
python -m app.scripts.create_admin
```

f) Lancer l'application

```
# Backend
uvicorn app.main:app --reload

# Frontend : Live Server sur Front-End/Home/home.html
```

PARTIE II : CONCEPTION

Afin de faciliter notre future implémentation et d'établir notre système d'information, nous sommes passés par une phase de réflexion très importante. Cela avait pour but de dans un premier temps définir l'environnement de notre système et dans un second temps garantir la fiabilité et la cohérence de notre environnement en évitant les erreurs de conception classique qui nous auraient retardé.

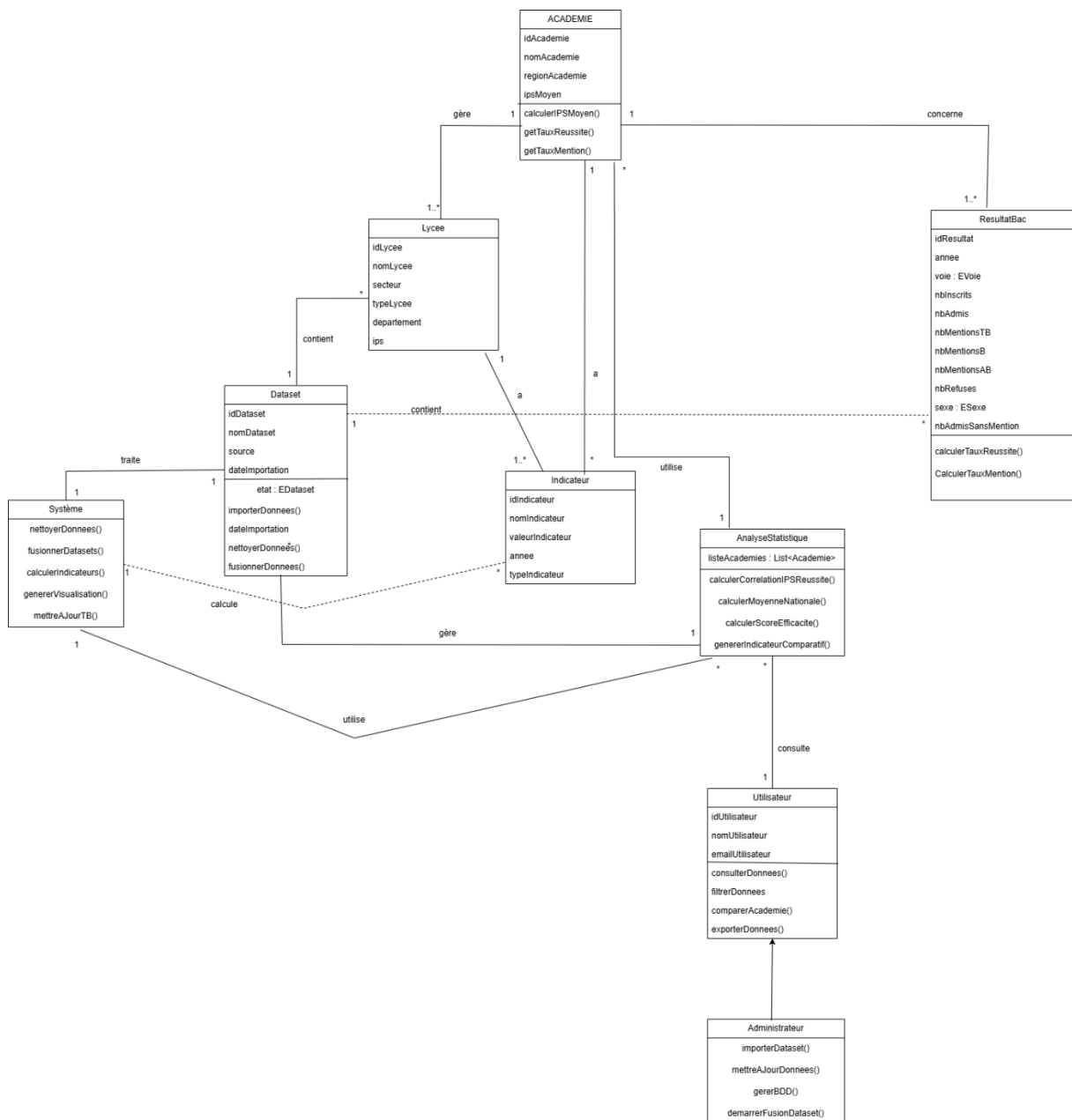
Pour cela, différents diagrammes ont été établi.

1. Diagramme de classe

a) Règles de gestion

N°	Règle de gestion	Type
R1	Un utilisateur peut accéder à l'application sans se connecter.	Traitement
R2	Un administrateur doit obligatoirement se connecter pour accéder aux fonctions d'administration.	Traitement
R3	Un utilisateur non-administrateur ne peut pas accéder aux fonctions d'administration.	Traitement
R4	Le système doit distinguer les utilisateurs administrateurs des autres utilisateurs.	Données
R5	Le système doit vérifier le rôle de l'utilisateur avant d'autoriser l'accès aux espaces sensibles.	Traitement

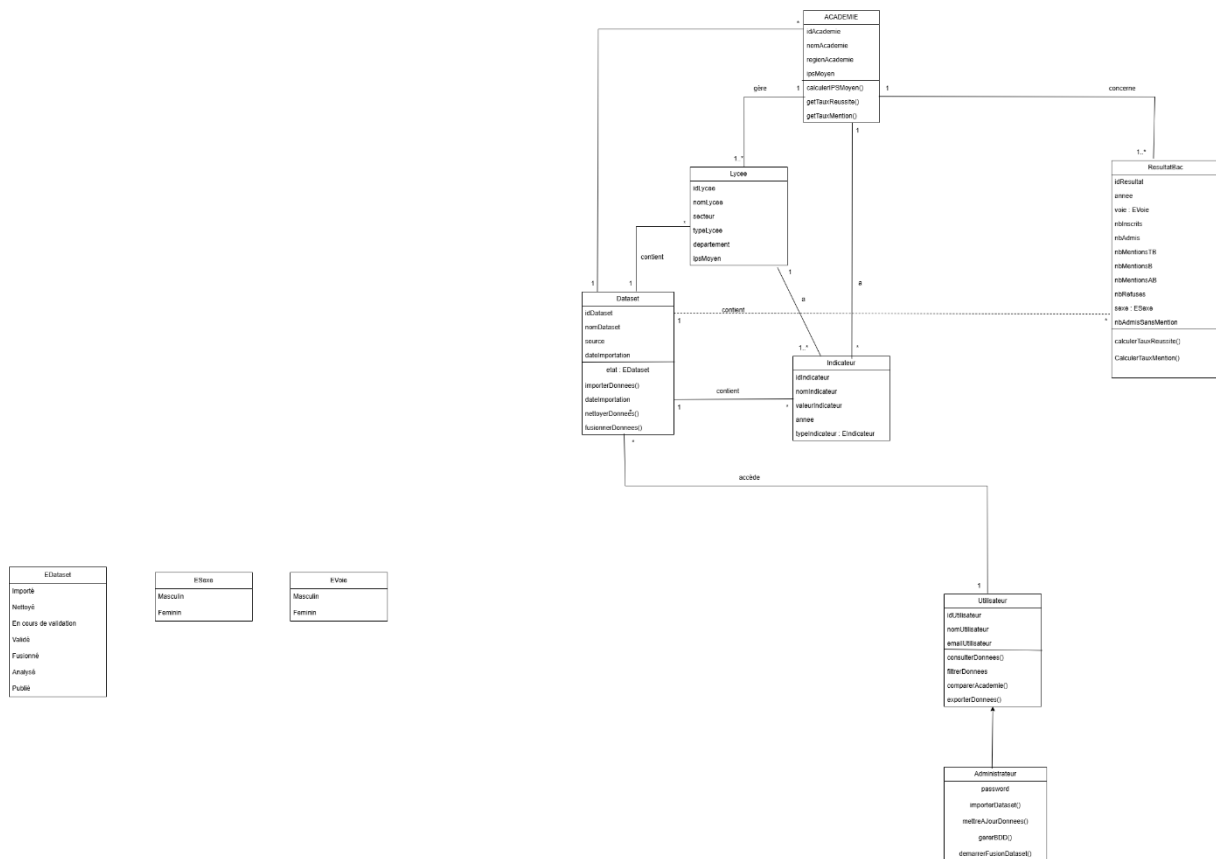
b) Diagramme de classe 1.0



Ceci est la première version de notre diagramme de classe. Ce diagramme comporte 8 classes qui seront les objets métiers de notre système. Cela dit, ce diagramme a évolué au fil de notre projet.

2. Evolution de la conception

a) Diagramme de classe 2.0



Comme dit précédemment, nous avons fait évoluer notre diagramme de classe initiale afin de fluidifier l'implémentation.

Le seul changement majeur à donc été de supprimer la classe « Système » car nous avons jugé que la création d'une instance du système n'est pas nécessaire et de ferait que complexifier l'implémentation. Les fonctionnalités liées au système seront donc directement implémentées dans le système d'information comme fonctionnalités automatique.

b) Dictionnaire de données

Entité	Attribut	Type	Description	Contrainte
Académie	idAcademie	INT	Identifiant unique de l'académie.	Clé primaire, non nul, auto-incrémenté
Académie	nomAcademie	VARCHAR	Nom de l'académie.	Non nul
Académie	regionAcademie	VARCHAR	Région académique associée.	Non nul
Académie	ipsMoyen	DECIMAL	IPS moyen calculé pour l'académie.	Valeur numérique, peut être calculée
Académie	calculerIPSmoyen()	Méthode	Calcule l'IPS moyen.	Fonction de traitement
Académie	getTauxReussite()	Méthode	Retourne le taux de réussite.	Fonction de traitement
Académie	getTauxMention()	Méthode	Retourne le taux de mentions.	Fonction de traitement
Lycée	idLycée	INT	Identifiant unique du lycée.	Clé primaire, non nul, auto-incrémenté
Lycée	nomLycée	VARCHAR	Nom du lycée.	Non nul

Lycée	secteur	VARCHAR	Secteur du lycée, public ou privé.	Valeurs contrôlées
Lycée	typeLycee	VARCHAR	Type de lycée.	Non nul
Lycée	departement	VARCHAR	Département de localisation.	Non nul
Lycée	ipsMoyen	DECIMAL	IPS moyen du lycée.	Valeur numérique
Dataset	idDataset	INT	Identifiant unique du jeu de données.	Clé primaire, non nul, auto-incrémenté
Dataset	nomDataset	VARCHAR	Nom du dataset.	Non nul
Dataset	source	VARCHAR	Source du dataset.	Non nul
Dataset	dateImportation	DATE	Date d'importation des données.	Non nul
Dataset	EDataset	ENUM	État du dataset.	Valeurs possibles : importé, nettoyé, en cours de validation, validé, fusionné, analysé, publié

Dataset	importerDonnées()	Méthode	Lance l'importation.	Fonction de traitement
Dataset	nettoyerDonnées()	Méthode	Nettoie les données importées.	Fonction de traitement
Dataset	fusionnerDonnées()	Méthode	Fusionne les datasets.	Fonction de traitement
Indicateur	idIndicateur	INT	Identifiant unique de l'indicateur.	Clé primaire, non nul, auto-incrémenté
Indicateur	nomIndicateur	VARC HAR	Nom de l'indicateur.	Non nul
Indicateur	valeurIndicateur	DECIMA L	Valeur calculée ou observée.	Non nul
Indicateur	annee	YEAR	Année de l'indicateur.	Non nul
Indicateur	typeIndicateur	ENUM	Type de l'indicateur.	Exemple : IPS, réussite, mention
Utilisateur	idUtilisateur	INT	Identifiant unique de l'utilisateur.	Clé primaire, non nul, auto-incrémenté
Utilisateur	nomUtilisateur	VARC HAR	Nom de l'utilisateur.	Non nul
Utilisateur	emailUtilisateur	VARC HAR	Adresse email de l'utilisateur.	Unique, non nul

Utilisateur	consulterDonnees()	Méthode	Permet de consulter les données.	Fonction de traitement
Utilisateur	filtrerDonnees()	Méthode	Permet de filtrer les données.	Fonction de traitement
Utilisateur	comparerAcademies()	Méthode	Compare plusieurs académies.	Fonction de traitement
Utilisateur	exporterDonnées()	Méthode	Exporte les données.	Fonction de traitement
Administrateur	password	VARC HAR	Mot de passe de l'administrateur.	Non nul, confidentiel
Administrateur	importerDataset()	Méthode	Importe un dataset.	Fonction de traitement
Administrateur	mettreAJourDonnées()	Méthode	Met à jour les données.	Fonction de traitement
Administrateur	gérerBDD()	Méthode	Gère la base de données.	Fonction de traitement
Administrateur	démarrerFusionDataset()	Méthode	Lance la fusion des datasets.	Fonction de traitement
RésultatBac	idResultat	INT	Identifiant unique du résultat du bac.	Clé primaire, non nul, auto-incrémenté

RésultatBac	annee	YEAR	Année de passage du bac.	Non nul
RésultatBac	voie	ENUM	Type de voie du bac.	Exemple : générale, technologique , professionnelle
RésultatBac	nbInscrits	INT	Nombre d'élèves inscrits.	Valeur positive
RésultatBac	nbAdmis	INT	Nombre d'élèves admis.	Valeur positive
RésultatBac	nbMentionsTB	INT	Nombre de mentions très bien.	Valeur positive
RésultatBac	nbMentionsB	INT	Nombre de mentions bien.	Valeur positive
RésultatBac	nbMentionsAB	INT	Nombre de mentions assez bien.	Valeur positive
RésultatBac	nbRefuses	INT	Nombre de candidats ref.	Valeur positive
RésultatBac	sexe	ENUM	Sexe concerné par l'indicateur.	Exemple : Homme/Femme

RésultatBac	nbAdmisSansMention	INT	Nombre d'admis sans mention.	Valeur positive
RésultatBac	calculerTauxReussite()	Méthode	Calcule le taux de réussite.	Fonction de traitement
RésultatBac	calculerTauxMention()	Méthode	Calcule le taux de mention.	Fonction de traitement
EDataset	état	ENUM	État d'avancement du dataset.	Importé, nettoyé, en cours de validation, validé, fusionné, analysé, publié
ESexe	valeur	ENUM	Sexe pour les données de répartition.	Masculin, Féminin
EVoie	valeur	ENUM	Voie du bac.	Masculin, Féminin dans ton schéma, mais à corriger si besoin

c) Choix effectués

Afin de compléter notre conception, nous avons effectué plusieurs choix cruciaux et pertinents. Parmi ses choix :

- La table « Indicateur » sera rempli après les calculs des indicateurs choisies. Elle permettra de consulter tous les indicateurs.
- Nous avons choisi de calculer des indicateurs clés qui permettraient aux utilisateurs de pouvoir faire leur analyse. Parmi ses métriques :

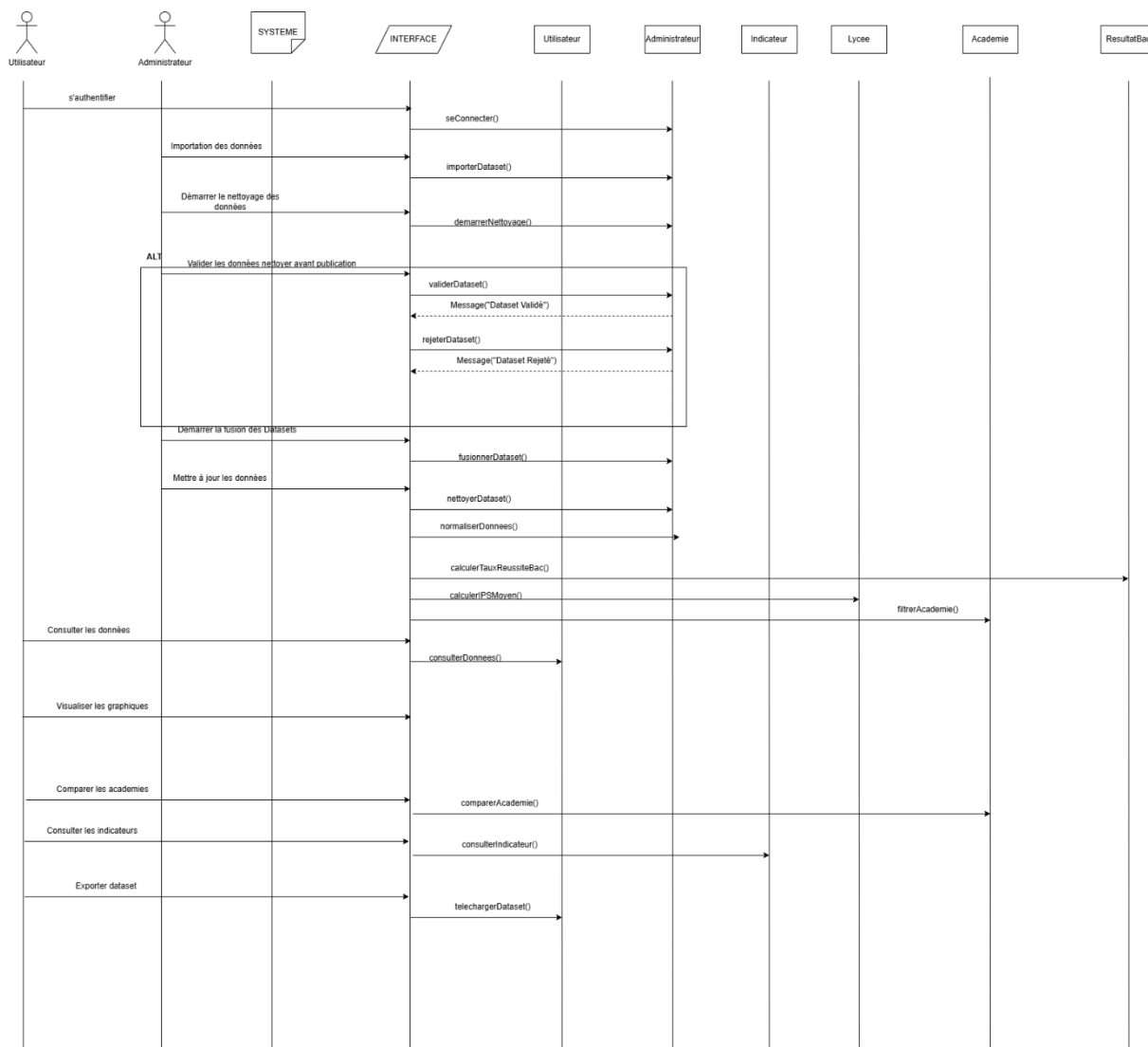
Indicateur	Formule	Type
Taux de réussite	$\text{nb_admis} / \text{nb_inscrits} \times 100$	%
Taux de mention	$(\text{nb_TB} + \text{nb_B} + \text{nb_AB}) / \text{nb_admis} \times 100$	%
Taux d'échec	$\text{nb_refuses} / \text{nb_inscrits} \times 100$	%
Taux réussite filles	$\text{admis_F} / \text{inscrits_F} \times 100$	%
Taux réussite garçons	$\text{admis_M} / \text{inscrits_M} \times 100$	%
Écart F/G	$\text{taux_F} - \text{taux_M}$	points
IPS moyen	AVG(ips) sur tous les lycées	float
Écart-type IPS	STDDEV_POP(ips) (mesure d'inégalité)	float
Nb académies/lycées	COUNT(*)	entier

Précision : l'écart-type IPS est calculé en SQL via `func.stddev_pop()` de PostgreSQL, pas en Python. Cela permet de traiter 7 200 lignes en quelques millisecondes.

3. Autres diagrammes pertinents

a) Workflow général de l'application

Dans le but de mieux comprendre et de visualiser le fonctionnement global de l'application, nous avons établi un workflow général en diagramme de séquence.



Scénario d'utilisation :

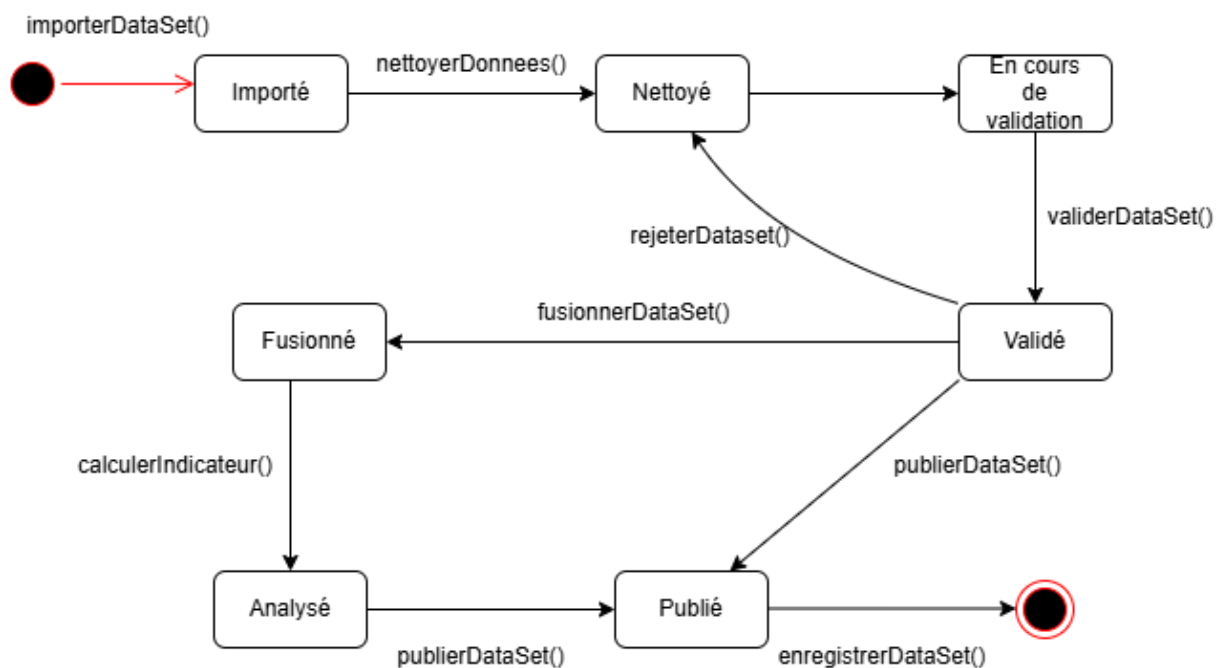
Le cycle de vie de la donnée sur la plateforme EduData s'articule autour d'une séparation claire entre la préparation technique et l'exploration visuelle. Tout commence en coulisses avec l'Administrateur, qui, après s'être authentifié en toute sécurité, importe les fichiers de données bruts (comme les résultats du Bac et l'IPS). Il peut lancer le nettoyage automatique puis valider visuellement le résultat ; si les données sont jugées conformes, il déclenche leur fusion et leur mise à jour. Cette action ordonne au système de

normaliser les informations et de calculer les indicateurs clés (tels que le taux de réussite par lycée ou l'IPS moyen etc.) afin de générer une base de données "maître" consolidée. Une fois ce travail de l'ombre terminé, l'Utilisateur final (parent, journaliste ou chercheur) entre en scène sur l'interface publique. Libéré de toute complexité technique, il navigue de manière fluide pour consulter les données à travers des graphiques interactifs, filtrer les informations selon ses besoins, ou encore utiliser le comparateur pour confronter directement les indicateurs de deux académies distinctes. Enfin, s'il souhaite poursuivre ses recherches de son côté, l'utilisateur clôture son parcours en exportant d'un simple clic ce jeu de données propre, validé et prêt à l'emploi.

b) Diagramme d'état-transition « Dataset »

Un seul objet de notre système avait un cycle de vie, c'est-à-dire qu'il changeait d'état en fonction des différentes actions des acteurs du système. Il s'agit de l'objet « Dataset ».

Les différents états de l'objet sont donc les suivants :



4. Challenges de conception

Les différents défis de notre conception ont été :

- Identifier les différents objets de notre système car nous n'avions pas de sujet ou de cahier de charge fourni par un client comme base de conception.

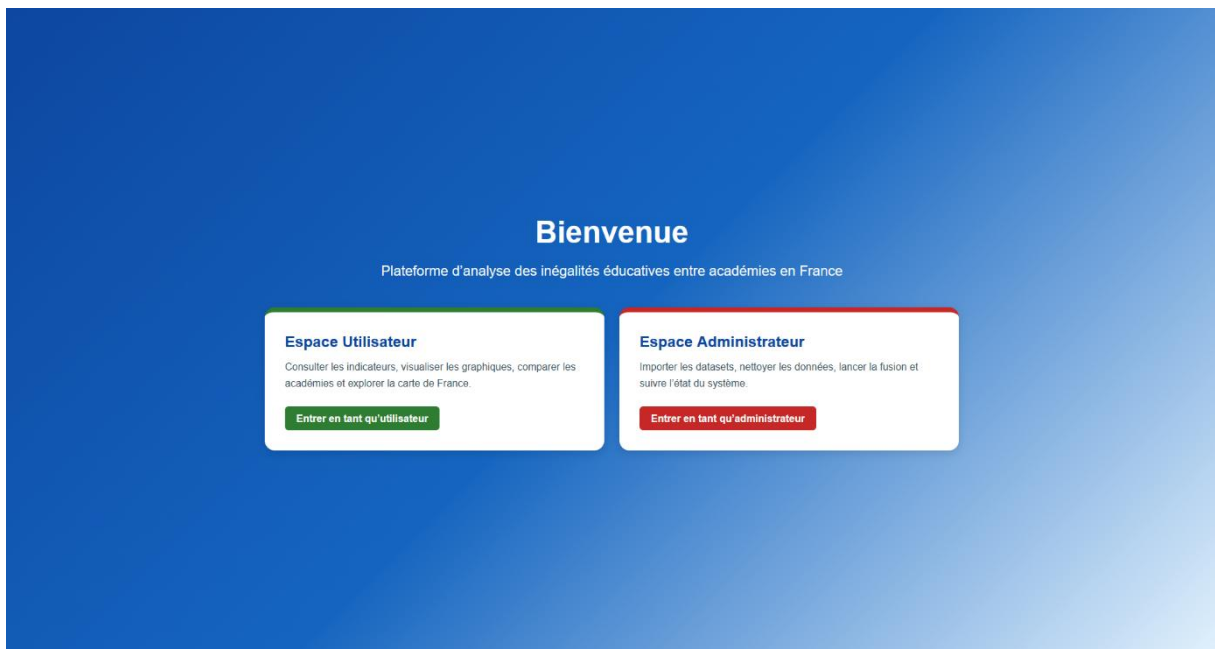
- Identifier les différentes relations entre les objets afin de les faire communiquer entre eux dans le système.
- Se mettre d'accord sur la cohérence des attributs de nos classe en fonction des données que nous avons.

5) Maquettes

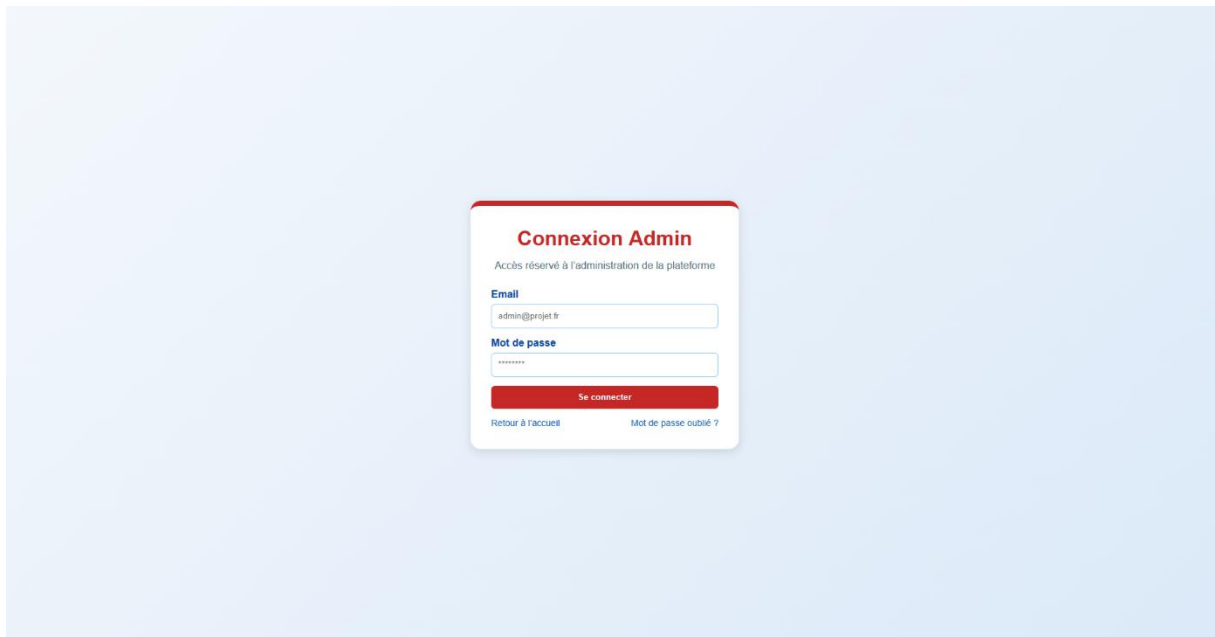
La fin de notre conception a donc porté sur l'élaboration de maquette pour notre plateforme afin d'avoir un visuel sur la future application que nous voulions implémenter.

Nous avons donc réalisé :

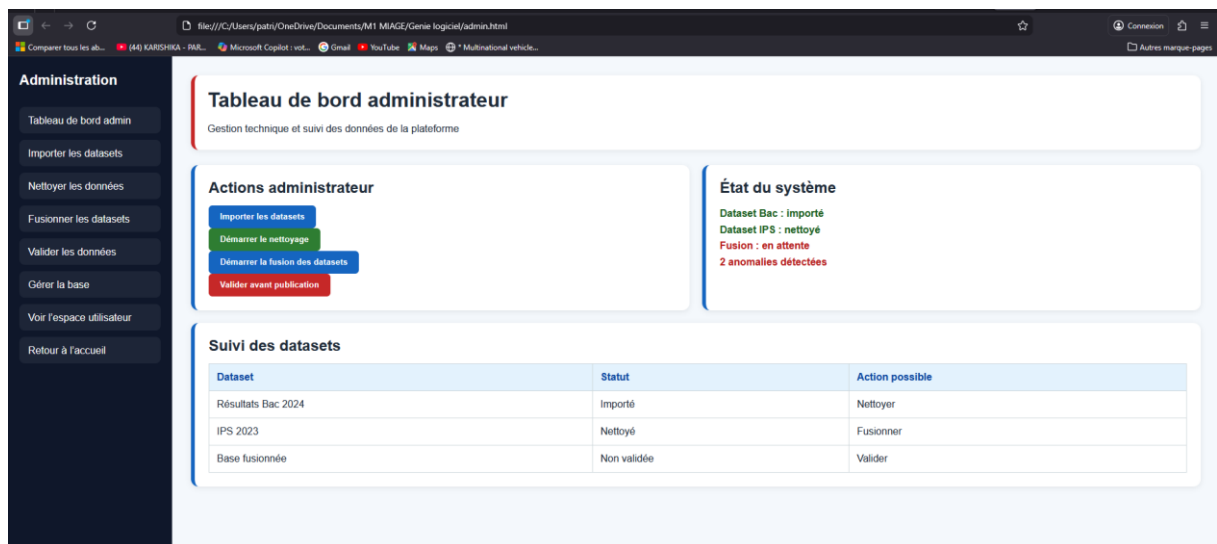
- **Une maquette pour notre page home :**



- **Une maquette de la page de connexion de l'administrateur :**



- Maquette du tableau de bord de l'administrateur :



- Maquette de la page de comparaison des académies (côté utilisateur) :

Comparaison des académies

Accueil
Comparaison
Carte
Indicateurs
Admin

Comparer deux académies

Choisir les académies et l'année pour comparer leurs résultats, leurs IPS et leurs écarts.

Académie 1

Académie 2

Année

Comparer

Résultats comparatifs

Indicateur	Académie 1	Académie 2
Taux de réussite	95 %	89 %
IPS moyen	112	98
Taux de mentions	42 %	31 %

Graphique comparatif

Zone pour un graphique Plotly ou Chart.js.

- Maquette de la page de la carte interactive (coté utilisateur) :

Carte interactive de la France

Accueil
Comparaison
Carte
Indicateurs
Admin

Filtres cartographiques

Indicateur affiché

Année

Afficher

Nous avons donc utilisé ces maquettes comme base de conception de notre Frontend. Cependant, nous avons effectué des modifications afin que le thème principal de la plateforme fasse plus plateforme d'analyse de données en adoptant le **thème vert** à l'instar des application homologues comme Excel pour ne citer que celle-là.

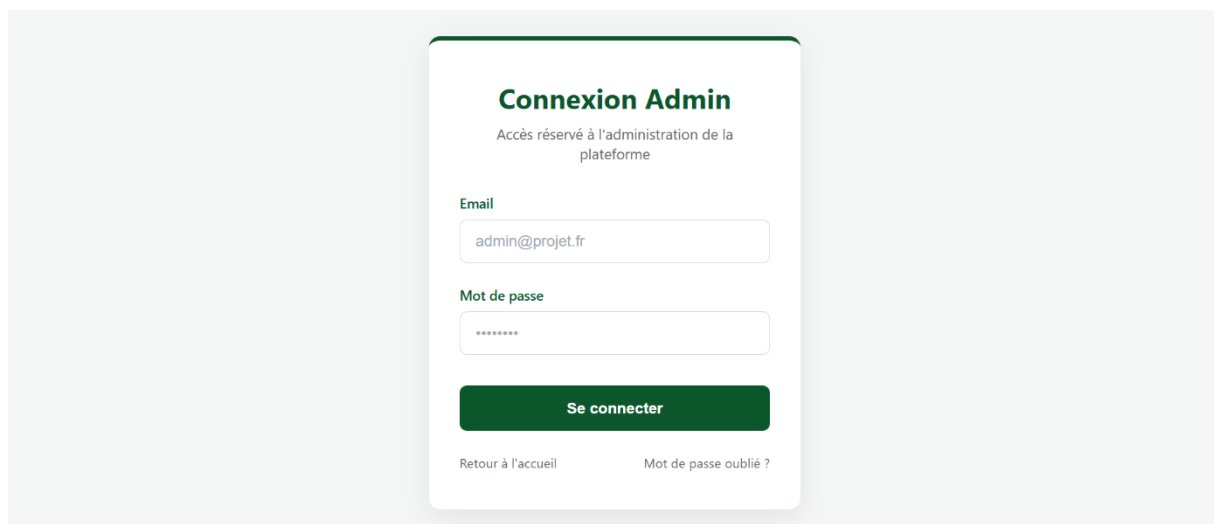
Voici un aperçu de quelques pages de notre plateforme :

- Page home :

22



- **Page de connexion :**



PARTIE III : DESCRIPTION TECHNIQUE

1. Technologies utilisées

Pour mettre sur pied notre application, nous avons choisi comme langage de programmation **Python**.

Nous avons donc utilisé plusieurs technologies python pour notre implémentation.

Framework web (Création de l'API) :

- **FastAPI** : C'est le cœur de notre application web. Il nous a servi de pont entre notre base de données et notre Frontend.

- **Unicorn** : C'est notre serveur web. FastAPI est le code de votre application, mais c'est Uvicorn qui se charge d'écouter les requêtes arrivant sur le port web et de faire tourner FastAPI.

Base de données (Stockage et interaction) :

- **SQLAlchemy** : C'est un ORM (Object-Relational Mapper) qui nous a permis d'interagir avec la base de données en utilisant des objets et du code Python pur sans avoir besoin d'écrire nos requêtes SQL complexes à la main.
- **Psycopg2-binary** : C'est notre pilote qui permet à Python de discuter avec notre base de données de type PostgreSQL.
- **Alembic** : C'est notre outil de migration utilisé pour ajouter des éléments à notre base de données comme des colonnes.

Validation & Config (Sécurité des données entrantes)

- **Pydantic** : C'est le partenaire privilégié de FastAPI. Il vérifie automatiquement que les données envoyées à votre API ont le bon format.
- **python-dotenv** : utilisé pour charger nos variables secrètes comme le mot de passe de l'administrateur.

Authentification (Connexion et sécurité)

- **bcrypt** : c'est la bibliothèque de cryptographie utilisée pour "hacher" les mots de passe.
- **python-jose** : utilisé pour générer et vérifier les "JWT" (JSON Web Tokens). C'est ce qui permet de maintenir la session ouverte quand un administrateur se connecte.
- **python-multipart** : utilisée pour traiter les données envoyées via des formulaires HTML.

Data processing (Traitement des données)

- **Pandas & Numpy** : sont les deux bibliothèques reines de la Data Science en Python. Elles nous ont servi à l'implémentation des fonctionnalités de l'administrateur telles que l'import des jeux de données.

Tests (Assurance qualité)

- **pytest** : c'est le Framework standard pour écrire et lancer des tests automatisés, afin de vérifier que vos fonctions Python marchent correctement.
- **httpx** : c'est un client HTTP (comme un navigateur virtuel) qui couplé à Pytest, nous a permis de simuler des appels à votre API FastAPI pour tester si elle renvoie bien les bonnes réponses.

2. Traitement des données

Afin de faciliter la manipulation de nos données, nous avons décidé de créer une base de données SQL pour notre application.

Pour cela, trois principales étapes ont été nécessaire :

- **Le nettoyage des jeux de données** : en effet nos jeux de données initiaux contenaient des informations qui soient n'étaient pas pertinentes, soient que nous pouvions calculer nous-même. On a donc procédé à un nettoyage des fichiers via un code Python, ce qui nous à rendu 03 fichiers csv propres à savoir « Academies.csv », « Lycees.csv » et « ResultatsBac.csv ».
- **La création de la base de données** : il va de soi qu'il nous a fallu établi un script SQL pour pouvoir créer la base de données qui accueillera nos données. Le choix du SGBD pour notre projet a été PostgreSQL pour notre familiarité avec d'un côté mais surtout pour sa propriété de gestion des droits. Le script de création est disponible **en annexe**.
- **L'import de nos données dans notre base** : l'étape finale. Après la création de notre base de données, il a fallu que l'on importe toutes nos données dans celle-ci. Cela a été possible grâce à des fonctionnalités de PostgreSQL qui le permettent. La principale difficulté a été de s'assurer de la cohérence et de l'ordre des colonnes de nos jeux de données avec les attributs de nos tables de la base afin d'éviter les erreurs et un mauvais remplissage.

3. Challenges techniques

Afin de mettre sur pied notre application, l'équipe a dû surmonter plusieurs défis techniques majeurs parmi lesquels :

- **Le traitement et la normalisation des données brutes** : concevoir un pipeline capable d'extraire, de transformer et de charger des fichiers CSV hétérogènes n'a pas été évident. Cela impliquait la gestion de différents formats, comme la détection automatique des séparateurs mais également fallu la normalisation des données de manière rigoureuse, la conversion des décimales au format français, la suppression des espaces superflus, l'harmonisation des noms d'académies en majuscules, nettoyage des lignes contenant des valeurs critiques vides etc. Pour faciliter cela, nous avons choisi nous-même des critères de nettoyage de base afin que de pouvoir contrôler le nettoyage : comme le choix de la suppression de colonnes non pertinentes.
- **L'optimisation des performances d'insertion et de calcul** : Le traitement d'un volume important de données, comprenant environ 7 200 lycées, a nécessité des

optimisations de performance. Pour accélérer le processus, l'équipe a utilisé la méthode `db.bulk_insert_mappings()` au lieu d'insertions ligne par ligne, rendant l'importation environ 50 fois plus rapide. De plus, pour calculer des indicateurs complexes tels que l'écart-type de l'IPS, le calcul a été délégué directement aux fonctions natives du moteur PostgreSQL (via `func.stddev_pop()`), ce qui permet de traiter l'ensemble des données en quelques millisecondes plutôt que de réaliser l'opération en Python.

- **La structuration de l'architecture logicielle backend** : un défi majeur a été de maintenir un code propre et évolutif en séparant correctement les responsabilités (Single Responsibility Principle). Le backend a donc été structuré en 5 couches distinctes : Client HTTP, FastAPI App, Routers, Couche CRUD, Couche Services et Modèles ORM. Cette conception offre un avantage significatif : une modification du code SQL n'affecte que la couche CRUD, sans risquer de casser les routes de l'API REST.
- **La sécurisation de l'application et de l'espace administrateur** : afin d'assurer la sécurité de l'application côté administrateur, l'équipe a utilisé le hachage *bcrypt*, intégrant un "salt" automatique et 12 "rounds" d'itération pour se prémunir contre les attaques par force brute. La protection des sessions s'est faite via des tokens JWT, et il a fallu configurer des dépendances *FastAPI* pour verrouiller l'accès aux routes d'administration, générant des erreurs 401 ou 403 pour les utilisateurs non autorisés.
- **Le développement d'un Frontend dynamique sans Framework** : le développement du Frontend a été réalisé en HTML/CSS avec du JavaScript "Vanilla" afin de rendre le tout plus dynamique, sans Framework comme React ou Vue. Cela a créé un défi technique pour la gestion de l'état global et des communications avec l'API. L'équipe a dû concevoir un module centralisé (*apiFetch()*) chargé d'injecter automatiquement le *token JWT* du *localStorage* dans les requêtes, de gérer les erreurs HTTP et de déclencher la déconnexion automatique en cas de token expiré.

PARTIE IV : APPLICATION

C'est ainsi que nous avons réussi à mettre sur pied « IDMC – EDUDATA » notre plateforme interactive d'analyse de données basé sur l'éducation et la corrélation entre les performances académiques et le niveau de vie des élèves. L'application se présente comme suit :

1. Présentation détaillée de l'application

L'application **IDMC - ÉduData France** est une plateforme web analytique conçue pour explorer, visualiser et comprendre les inégalités éducatives à travers le prisme de l'Indice de Position Sociale (IPS) et des résultats du baccalauréat. Afin de garantir des performances optimales et une expérience fluide, l'architecture repose sur une séparation stricte des responsabilités entre le traitement lourd des données (Backend/Administration) et leur restitution visuelle (Frontend/Utilisateur).

L'application se divise en deux environnements distincts, répondant chacun à des cas d'usage précis.

a) L'Espace Administrateur :

Cet espace sécurisé est le cœur technique de la plateforme. Il est réservé à l'équipe technique chargée de maintenir la qualité et l'intégrité de la base de données. L'accès y est protégé par un système d'authentification robuste. C'est la page « Home » de l'application. L'utilisateur lambda s'il est administrateur pourra donc se connecter à son espace de travail via cette page.

Une fois authentifié, l'administrateur dispose d'un tableau de bord complet agissant comme tour de contrôle pour orchestrer l'environnement de la plateforme :

- **Importation des jeux de données** : cette interface offre une zone de glisser-déposer intuitive permettant l'ingestion de fichiers bruts (CSV ou Excel) issus de l'Open Data gouvernemental. Le système détecte automatiquement la structure des fichiers importés.
- **Nettoyage et Normalisation** : cette vue permet une intervention fine sur la qualité des données. L'administrateur peut visualiser un échantillon des données brutes et configurer des règles de traitement via Pandas : suppression des doublons exacts, traitement des valeurs manquantes, suppression de colonnes superflues et normalisation des chaînes de caractères (Trim, majuscules). Un aperçu du résultat post-nettoyage valide l'opération avant persistance.
- **Fusion des Données** : C'est la fonctionnalité critique de l'ingénierie de la plateforme. Elle permet de réaliser des jointures relationnelles complexes (par exemple, croiser les taux de réussite avec l'IPS via le Code UAI de l'établissement). Cette étape génère la table consolidée ("Master Table") qui servira de source unique de vérité pour le front-office, évitant ainsi des calculs coûteux lors de la consultation publique.

b) L'Espace Utilisateur :

Cet espace public est conçu pour une consommation fluide, rapide et interactive de l'information. L'utilisateur (chercheur, journaliste ou citoyen) accède à une interface moderne et épurée, sans aucune friction technique, la donnée ayant déjà été traitée en amont.

La navigation s'articule autour d'un menu horizontal permanent desservant cinq vues principales :

- **Le Tableau de Bord Central** : Une page d'atterrissage présentant des KPI nationaux (Taux de réussite global, IPS moyen, nombre d'académies). Une barre de filtres globaux (Année, Académie, Indicateur) permet de restreindre dynamiquement le périmètre des données analysées.
- **Exploration Spatiale** : Une interface cartographique interactive (implémentée via Leaflet) divise l'écran entre un panneau de contrôle à gauche et la carte de France à droite. L'utilisateur peut y visualiser la répartition géographique de l'IPS ou des taux de réussite à l'aide d'échelles choroplèthes, et obtenir des statistiques détaillées par académie au clic.
- **Analyses & Graphiques** : Un module de visualisation de données ("*DataViz*") généré côté client (via des bibliothèques comme *Chart.js*). Il propose plusieurs niveaux de lecture :
 - Des nuages de points (Scatter Plots) pour mettre en évidence les corrélations (ex: impact de l'IPS sur la réussite).
 - Des diagrammes en barres pour établir des classements (Top/Flop des académies).
 - Des courbes temporelles (Line Charts) pour suivre l'évolution des indicateurs sur plusieurs années.
- **Le Comparateur d'Académies** : Une fonctionnalité d'analyse comparative directe (A/B). L'utilisateur sélectionne deux académies distinctes pour afficher un face-à-face visuel de leurs indicateurs clés grâce à des barres de progression proportionnelles, facilitant l'identification des disparités régionales.
- **Le Catalogue Open Data** : Pour garantir la transparence et encourager la recherche indépendante, cette page recense les jeux de données bruts et consolidés. Elle informe sur les licences (Etalab) et propose le téléchargement direct des fichiers CSV avec leurs métadonnées associées.

2. Perspectives

Cela dit, notre application a donc des axes d'améliorations surtout via des fonctionnalités qui avaient été prévues dès le départ comme :

- **Fusion des jeux de données** : nécessite une refonte du schéma pour stocker les jointures.
- **Niveau "Académie" sur l'accueil** : actuellement filtre par année uniquement.
- **Validation manuelle des imports** : actuellement automatique.
- **Mise à jour automatique des données** : actuellement manuelle via les CSV.

PARTIE V : CONCLUSION

La réalisation du projet **IDMC - ÉduData France** a permis de concevoir et de déployer une plateforme web analytique robuste, répondant concrètement à la problématique des inégalités éducatives en France. En croisant des jeux de données ouverts et massifs, ce projet démontre l'intérêt de l'ingénierie logicielle au service de l'analyse citoyenne.

Sur le plan de la conception, nous avons privilégié une architecture modulaire et performante. La séparation stricte entre le moteur de traitement des données (en arrière-plan) et l'interface de restitution visuelle a permis de garantir la fiabilité et la rapidité du système, même lors de la manipulation de grands volumes d'informations.

Cette approche rigoureuse s'est accompagnée d'une forte orientation utilisateur. La création d'une double interface — un espace d'administration hautement sécurisé pour la gestion des données, et un espace public intuitif offrant des visualisations interactives — permet de répondre précisément aux besoins d'utilisateurs très différents.

En définitive, bien que certaines évolutions fonctionnelles soient envisageables à l'avenir, le socle applicatif livré est complet, testé et opérationnel. Ce projet constitue ainsi une expérience formatrice majeure, consolidant nos compétences transversales en conception de systèmes d'information et en gestion de projet, en parfaite adéquation avec les exigences de notre cursus en Master MIAGE.

Annexes

Les liens vers les sources de données et les différentes images de diagrammes et les captures d'écran sont disponibles dans le fichier « Annexe ».